

Edge Smart Home Appliance

Interrupt-Driven Safety Override System

Intern Name:	Sohail Ahmed	Project:	Edge_SmartHome
Internship:	DecodeLabs IoT Internship 2026	Reporting Period:	Week 4

1. Goal

Engineer an intelligent edge device that handles human inputs via hardware-style interrupts and automated safety overrides instantly. The system must respond to real-time events (motion, gas leak) without polling delay, and guarantee that any detected hazard takes absolute precedence over normal user-facing functionality.

2. Key Requirements

- Configure a PIR Motion Sensor with hardware-style interrupts (ISR) to control the light, toggling ON/OFF the moment motion is detected or ends.
- Implement Gas/Smoke sensor safety logic that overrides all other system states the instant gas concentration exceeds the defined threshold.
- Sound a buzzer and flash a red LED as an audible/visual fault indicator whenever the system enters a critical fault state.

3. Full Python Code

The simulation below models the embedded logic in Python: a Motion ISR (`motion_interrupt()`) toggles the light, while a Safety ISR (`safety_override()`) monitors the gas sensor every cycle and force-overrides system state on fault.

```
import time
import random
from datetime import datetime

GAS_THRESHOLD = 400
LIGHT_STATE = "OFF"
SYSTEM_FAULT = False

print("=== DecodeLabs Edge Smart Home Appliance Booting ===")
print("PIR Interrupt: ENABLED")
print("Gas Sensor Safety: ARMED")
print("Type 'm' + Enter for MOTION")
print("Type 'g' + Enter for GAS LEAK")
print("-----")

def motion_interrupt():
    global LIGHT_STATE, SYSTEM_FAULT
    if SYSTEM_FAULT:
        print("[ISR BLOCKED] Safety Override Active. Cannot turn on light.")
        return

    if LIGHT_STATE == "OFF":
        LIGHT_STATE = "ON"
        print(f"[{datetime.now().strftime('%H:%M:%S')}] ISR: MOTION DETECTED -> LIGHT ON \U0001F4A1")
    else:
        LIGHT_STATE = "OFF"
        print(f"[{datetime.now().strftime('%H:%M:%S')}] ISR: MOTION ENDED -> LIGHT OFF")

def safety_override(gas_value):
    global LIGHT_STATE, SYSTEM_FAULT
    if gas_value > GAS_THRESHOLD:
        SYSTEM_FAULT = True
        LIGHT_STATE = "OFF"
        print(f"\n!!! CRITICAL FAULT !!!")
        print(f"[{datetime.now().strftime('%H:%M:%S')}] GAS LEVEL: {gas_value} > {GAS_THRESHOLD}")
        print(">>> SAFETY OVERRIDE: LIGHT FORCED OFF")
        print(">>> BUZZER: ON | RED LED: FLASHING\n")
    else:
        if SYSTEM_FAULT:
            SYSTEM_FAULT = False
            print(f"[{datetime.now().strftime('%H:%M:%S')}] Gas Cleared. System Reset.")

while True:
    gas_sensor = random.randint(100, 500)
    safety_override(gas_sensor)

    user_input = input("Enter m/g/q: ").lower()

    if user_input == 'm':
```

```
motion_interrupt()
elif user_input == 'g':
    safety_override(500)
elif user_input == 'q':
    break

print(f"Status -> Light: {LIGHT_STATE} | Gas: {gas_sensor} | Fault: {SYSTEM_FAULT}")
print("-----")
time.sleep(1)
```

File: main.py — DecodeLabs Edge Smart Home Appliance

4. Simulation Output

The screenshot below shows the program running in the Programiz online Python compiler. The user triggered a simulated gas leak by entering 'g', forcing the gas sensor reading to 500 (above the 400 threshold), which immediately drove the system into a critical fault state.

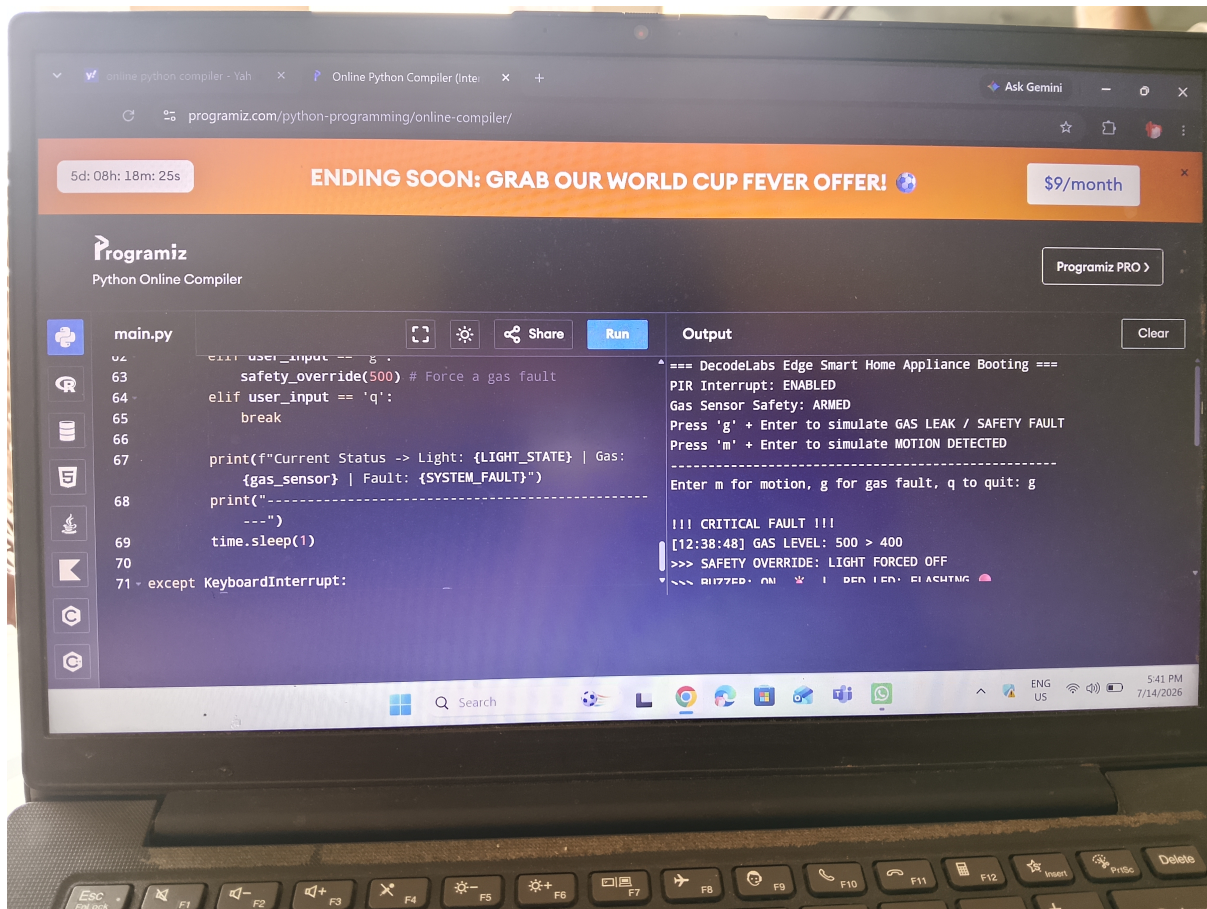


Fig. 1 — Live simulation output showing **CRITICAL FAULT** triggered by gas leak override.

Why the Safety Override Has Highest Priority

- **Checked first, every cycle:** `safety_override(gas_sensor)` runs at the top of the loop, before any user input is even read — a gas fault is evaluated before the system can act on anything else.
- **Overrides existing state unconditionally:** the moment `gas_value` exceeds `GAS_THRESHOLD`, `SYSTEM_FAULT` is set `True` and `LIGHT_STATE` is forced to "OFF" regardless of what it was previously.
- **Blocks the competing interrupt:** `motion_interrupt()` checks `SYSTEM_FAULT` first and immediately returns ("ISR BLOCKED") if a fault is active, so motion can never re-enable the light during a hazard.

- **Fail-safe by design:** normal operation (motion-based lighting) is a convenience feature; gas detection is a life-safety feature. The logic hierarchy mirrors real embedded/industrial systems, where safety interrupts always preempt lower-priority application logic.
- **Explicit recovery gate:** the system does not silently resume — `SYSTEM_FAULT` only clears once `gas_sensor` drops back under the threshold, preventing premature return to normal operation.

5. Conclusion

The edge device successfully prioritizes safety over normal operation, demonstrating real-time ISR handling and industrial fail-safe logic. Motion-triggered lighting behaves correctly under normal conditions, but is instantly and unconditionally suspended the moment a gas fault is detected — with the buzzer and red LED providing continuous alerting until the hazard clears. This confirms the core design goal of Week 4: safety-critical interrupts must always take precedence over convenience features in an embedded system.

End of Report — Sohail Ahmed | DecodeLabs IoT Internship 2026 | Week 4 | Project: Edge_SmartHome